

TP1 - Programmation Orientée Objet en TypeScript

Exercice 1 :

Créer la classe `Book` avec les attributs publics suivants :

- `title: string`
- `author: string`
- `year: number`
- `pageCount: number`

Le constructeur reçoit ces 4 paramètres dans l'ordre indiqué.

Ajoutez une méthode publique `toString: string` qui renvoie une chaîne lisible regroupant toutes les informations d'une instance.

"The Pragmatic Programmer" (1999) by Andrew Hunt - 352 pages

Exercice 1.2 :

Passer l'ensemble des attributs en privé. Noter alors ce qu'il se produit lorsque vous essayez d'accéder à un des attributs d'un objet `Book` en dehors de la classe.

Mettre les attributs en privé permet d'empêcher leur lecture / modification en dehors de sa propre classe.

Pour laisser les utilisateurs modifier ces attributs, nous allons mettre en place des getters / setters sur le titre. Ce sont des méthodes publiques permettant d'accéder aux informations de la classe depuis l'extérieur de celle-ci.

Exercice 2 :

Créer la classe `Author` avec les attributs privés suivants :

- `firstName: string`
- `lastName: string`
- `nationality: string` (optionnel)

Le constructeur reçoit ces 3 paramètres dans l'ordre indiqué. Si le constructeur ne reçoit pas `nationality`, donner la valeur par défaut 'unknown'.

Créer une méthode publique `getFullName: string` qui retourne le nom complet d'un auteur.

Exercice 2.1 :

Modifier `Book` pour remplacer `author: string` par `author: Author`.

Étant donné que `author` n'est plus de type `string`, on ne peut plus l'afficher tel quel dans notre méthode `toString`. On peut cependant accéder à la méthode `getFullName` de notre objet `author`, comme celle-ci est publique.

Adapter la méthode `toString` de la classe `Book` en conséquence.

Exercice 3 :

Créer la classe `Library` avec :

- `books: Book[]`

Cette classe possède un constructeur ne prenant pas d'argument, `books` est initialisé avec un tableau vide.

Créer la méthode publique `addBook: void`, prenant en paramètre un objet de type `Book`, et qui l'ajoute dans le tableau de livres de la classe `Library`.

Exercice 3.1 :

Renommer la méthode `addBook` en `addBooks`.

Implémenter la surcharge de méthode sur `addBooks`. Cette méthode doit désormais également prendre en charge les tableaux de livres que nous lui fournissons.

Exercice 3.2 :

Implémenter la méthode statique `isLibraryOpen: boolean`, qui doit retourner un booléen indiquant si la bibliothèque est ouverte ou non (ouverte tous les jours sauf le dimanche).

Une méthode statique est une méthode ne dépendant pas de l'objet courant, mais appartenant à l'ensemble de la classe. Le mot clé `this` représente donc la classe en elle-même et pas uniquement une instance.

Exercice 4 :

Implémenter un type `Query` avec :

- `authorName: string`

- `order?: 'asc'|'desc'` (optionnel)
- `limit?: number` (optionnel)

Un type est une manière de définir la structure d'une donnée : quels champs elle contient, leurs noms, les types attendus et si certains champs sont optionnels. On peut ensuite utiliser notre type comme n'importe quel autre. Ainsi, on saura qu'un type `Query` aura toujours une même structure.

Exercice 4.1 :

Créer la méthode `getBooksByAuthorName: Book[]`

Celle-ci prendra en paramètre un objet `Query`, et retournera la liste des livres que l'auteur aura écrit.

Appeler la méthode `toString` sur les livres retournés pour avoir un affichage propre.